משימת תרגול #C - מחלקות, List ו-Dictionary

ניהול חנות ספרי קומיקס (Comic Book Store)

כיתה י"א | שפת #C | .NET 9.0 | בדיקות יחידה אוטומטיות

📚 חלק א': דף עזר ומילון מתודות (Cheat Sheet)

1. עבודה עם `Dictionary<TKey, TValue>` (מילון מפתח-ערך)

מילון הוא אוסף של זוגות **מפתח (Key)** ו**ערך (Value)**. המפתחות במילון חייבים להיות ייחודיים.

מתודה / מאפיין	תיאור	דוגמת שימוש בקוד #C
יצירת מילון	יצירת מילון חדש שבו המפתח מחרוזת והערך מספר שלם	`Dictionary<string, int> stock = new Dictionary<string, int>();`
`Add(key, value)`	הוספת זוג חדש למילון (זורק שגיאה אם המפתח כבר קיים)	`stock.Add("Spider-Man #300", 5);`
אינדקסר `[key]`	גישה, הוספה או עדכון ערך לפי מפתח (בטוח לשימוש)	`stock["Spider-Man #300"] = 10; // מעדכן או מוסיף`
`ContainsKey(key)`	בדיקה האם מפתח קיים במילון (מחזיר `bool`)	`if (stock.ContainsKey("Batman")) { ... }`
`ContainsValue(val)`	בדיקה האם ערך קיים במילון	`bool hasZero = stock.ContainsValue(0);`
`TryGetValue(...)`	שליפת ערך בצורה בטוחה ללא שגיאה במידה והמפתח לא קיים	`if (stock.TryGetValue("Batman", out int qty)) { Console.WriteLine(qty); }`

מתודה / מאפיין	תיאור	דוגמת שימוש בקוד #C
`Remove(key)`	הסרת מפתח והערך שלו מהמילון	`stock.Remove("Spider-Man #300");`
`Count`	החזרת מספר הזוגות במילון	`int totalKeys = stock.Count;`
`Keys` / `Values`	גישה לאוסף המפתחות בלבד או הערכים בלבד	`foreach (string title in stock.Keys) { ... }`
לולאת `foreach`	סריקת כל זוגות המפתח-ערך במילון	`foreach (KeyValuePair<string, int> pair in stock) { Console.WriteLine($"{pair.Key} -> {pair.Value}"); }`

2. עבודה עם `List<T>` (רשימה דינמית)

רשימה היא אוסף מסודר של איברים מאותו סוג, שגודלה משתנה באופן דינמי.

מתודה / מאפיין	תיאור	דוגמת שימוש בקוד #C
יצירת רשימה	יצירת מופע חדש של רשימה דינמית	`List<ComicBook> books = new List<ComicBook>();`
`Add(item)`	הוספת איבר לסוף הרשימה	`books.Add(new ComicBook("X-Men #1", 1991, "X-Men", 8));`
`Count`	החזרת מספר האיברים ברשימה	`int count = books.Count;`
אינדקס `[i]`	גישה או עדכון איבר במיקום מסוים (החל מ-0)	`ComicBook first = books[0];`
`Remove(item)`	הסרת המופע הראשון של האיבר מהרשימה	`books.Remove(comicObj);`
`RemoveAt(index)`	הסרת איבר לפי מיקום אינדקס	`books.RemoveAt(2); // מסיר איבר באינדקס 2`
`Contains(item)`	בדיקה האם איבר קיים ברשימה	`bool exists = books.Contains(comicObj);`

מתודה / מאפיין	תיאור	דוגמת שימוש בקוד #C
`Find(predicate)`	מציאת האיבר הראשון העונה לתנאי (או `null`)	`ComicBook b = books.Find(x => x.Title == "Batman");`
`Clear()`	מחיקת כל האיברים מהרשימה	`books.Clear();`

לולאת `foreach`	סריקת כל האיברים ברשימה לפי סדרם	`foreach (ComicBook b in books)`
		`{`
		`  Console.WriteLine(b.Title);`
		`}`

🛠️ חלק ב': תיאור הפרויקט והמחלקות

הפרויקט בנוי משתי מחלקות ראשיות בתוך מרחב השמות `ComicBookStore`:

- **`ComicBook`**: מייצגת חוברת קומיקס יחידה בחנות עם התכונות: `Title` (שם החוברת), `Year` (שנת הוצאה), `Hero` (שם הגיבור), ו-`StockQuantity` (כמות במלאי).
- **`ComicStore`**: מייצגת את החנות. מכילה את המאפיין `List<ComicBook> Inventory`. בפעולה הבונה (Constructor) שלה מאותחלת רשימה של **20 חוברות קומיקס שונות** (כולל חוברות שהמלאי שלהן הוא 0).

📝 חלק ג': המשימות לפתרון התלמידים

עליכם לממש את הפעולות הבאות במחלקה `ComicStore.cs`:

משימה 1: `AddOrUpdateComic(ComicBook newComic)`

הפעולה מקבלת עצם של חוברת קומיקס חדשה. יש לבדוק האם חוברת בעלת **אותו שם (Title)** כבר קיימת ברשימת `Inventory`:

- אם החוברת קיימת – יש להגדיל את הכמות במלאי (`StockQuantity`) של החוברת הקיימת בכמות של החוברת החדשה.
- אם החוברת אינה קיימת – יש להוסיף את החוברת החדשה לרשימה `Inventory`.

משימה 2: `UpdateStock(string title, int newQuantity)`

הפעולה מקבלת שם חוברת קומיקס וכמות חדשה למלאי. יש לחפש את החוברת ברשימה לפי שמה:

- אם החוברת נמצאה – יש לעדכן את כמות המלאי שלה ל-`newQuantity` ולהחזיר `true`.
- אם החוברת לא נמצאה – יש להחזיר `false`.

משימה 3: `SearchByHero(string heroName)`

הפעולה מקבלת שם של גיבור (למשל `"Batman"`). הפעולה מחזירה רשימה חדשה (`List<ComicBook>`) הכוללת את כל חוברות הקומיקס של אותו גיבור שנמצאות בחנות.

משימה 4: `IsInStock(string title)`

הפעולה מקבלת שם חוברת קומיקס. הפעולה מחזירה `true` אך ורק אם החוברת קיימת בחנות **ומהכמות במלאי שלה גדולה מ-0**. בכל מקרה אחר (אם המלאי 0 או שהיא לא קיימת) יש להחזיר `false`.

משימה 5: `GetStockDictionary()`

הפעולה אינה מקבלת פרמטרים. עליכם ליצור ולהחזיר מילון (`Dictionary<string, int>`) שבו:

- **המפתח (Key):** שם חוברת הקומיקס (`Title`).
- **הערך (Value):** הכמות שלה במלאי (`StockQuantity`).

משימה 6: `GetQuantityFromDictionary(string title)` חובה!

הפעולה מקבלת שם חוברת קומיקס.
חובת מימוש: בתוך הפעולה יש לזמן קודם כל את הפעולה `GetStockDictionary()` שמימשתם בסעיף 5!

לאחר מכן, יש לחפש במילון המתקבל את שם החוברת:

- אם החוברת קיימת במילון – החזירו את הכמות שלה במלאי.
- אם החוברת אינה קיימת במילון – החזירו `-1`.

משימה 7 (אתגר בונוס): `GetHeroWithMostComics()`

הפעולה מחזירה את שם הגיבור שיש לו הכי הרבה חוברות קומיקס שונות בחנות (ללא קשר לכמות המלאי של כל חוברת).

> 💡 **רמז:** מומלץ להיעזר במילון עזר שסופר כמה חוברות קיימות לכל גיבור (מפתח: שם הגיבור, ערך: כמות החוברות).

חלק ד': הרצת בדיקות אוטומטיות והנחיות הגשה 🧪

בתוך הקובץ `Program.cs` מוגדרת מערכת בדיקות אוטומטית (Unit Tests). בלחיצה על מקש ההרצה (F5 / Run), התוכנית תבדוק את הפעולות שכתבתם ותדפיס טבלה צבעונית המסכמת אילו משימות עברו בהצלחה (`[V]`) ואילו נכשלו (`[X]`).

קריטריון להצלחה: להגיע לתוצאה של `7 / 7 משימות עברו בהצלחה!` 🏆.